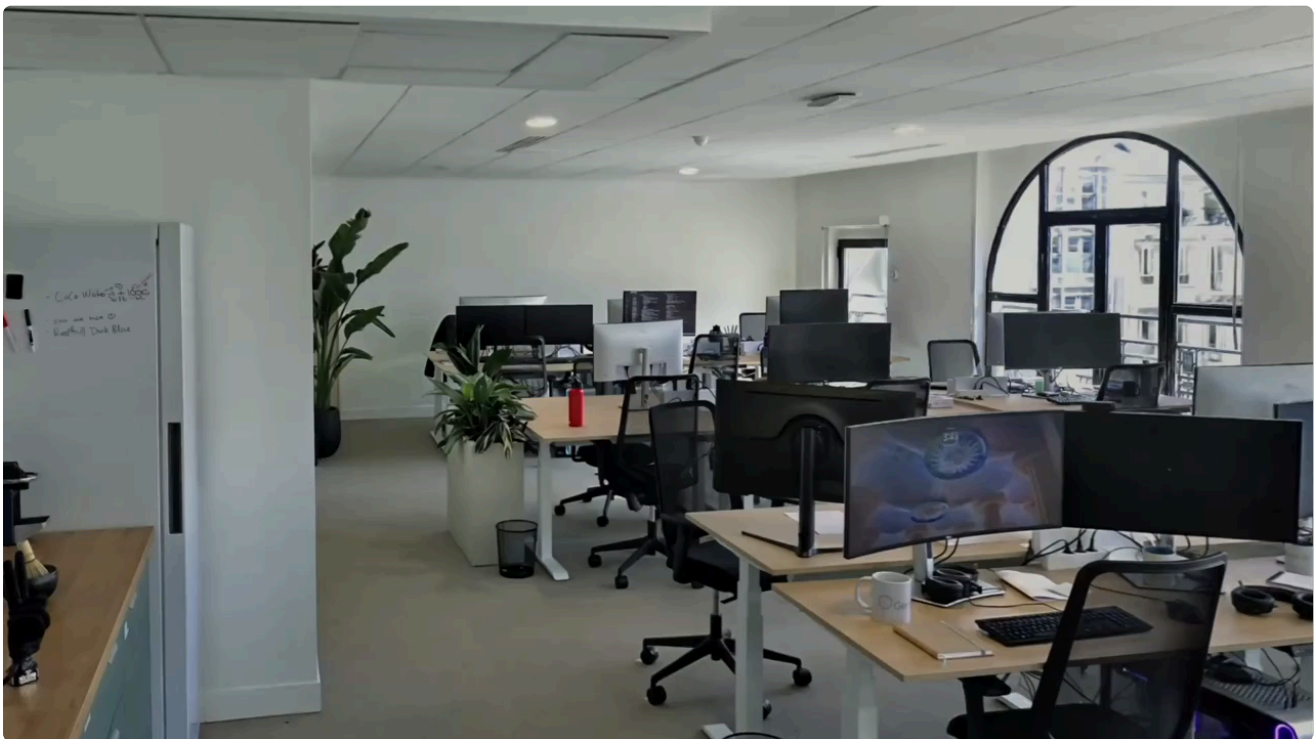


Роль моделирования в масштабируемой робототехнике, Genesis World 1.0 и дальнейшие перспективы



Моделирование — основа масштабируемой робототехники.

Автор : Команда Genesis

В Genesis мы подходим к робототехнике как к системной проблеме. Маховик данных — это основа для создания более качественного продукта, но одних ингредиентов недостаточно

для построения системы: нам также нужна надежная и эффективная инфраструктура для их обработки и поддержки научно-исследовательских и опытно-конструкторских работ.

Моделирование — один из важнейших элементов этой инфраструктуры. Оно десятилетиями играло ключевую роль в робототехнике, помогая ускорить развитие роботизированных систем в области управления, планирования и обучения моделей. До основания компании Genesis AI мы выпустили первую версию [Genesis World](#) (первоначальное название — Genesis) — первую попытку объединить несколько физических решателей, инструментов для моделирования и генерации данных в единую платформу с оптимизированной производительностью и удобным интерфейсом.

За последний год мы расширили первоначальную версию Genesis World, превратив ее в более систематизированную платформу, обеспечивающую реалистичность и производительность на всех этапах. В результате мы получили целостную инфраструктуру, включающую в себя оптимизированный компилятор с ускорением на графическом процессоре, новые методы решения задач без проникновения, унифицированную систему моделирования и высокопроизводительный конвейер фотореалистичного рендеринга.

В этом посте мы хотели бы поделиться с сообществом робототехников своими мыслями о развитии симуляционного моделирования: как, по нашему мнению, оно может ускорить исследования в области робототехники сегодня и как его роль может измениться в ближайшие годы.

💡 TL;DR

- Мы рассматриваем симуляцию не только как генератор данных, но и как механизм оценки и итераций для наших базовых моделей робототехники. Эксперименты в реальных условиях ограничивают скорость, с которой мы можем оценивать возможные решения, а симуляция снимает это ограничение, превращая цикл разработки из *проблемы, связанной с затратами времени в проблему, связанную с вычислительными ресурсами*.
- Наша оценка на основе симуляции тесно связана с тестированием на реальных устройствах, что позволяет проводить систематическую оценку в замкнутом цикле по всему комбинаторному пространству задач без использования смоделированных данных.
- Genesis World 1.0 — основа нашей инфраструктуры симуляции. Он включает в себя [Nyx](#), рендерер с трассировкой пути в реальном времени, специально разработанный для робототехники; физическую платформу [Genesis World](#), объединяющую моделирование жёстких и деформируемых тел с новым внешним ограничением подвижности для межсоединений и бесбарьерным решателем эластодинамики; а также [Quadrants](#), компилятор Python для GPU, оптимизирующий вычисления при моделировании и ускоряющий их с помощью графического процессора.

Моделирование ускоряет цикл разработки робототехнических моделей

Данные — самое известное узкое место в робототехнике, и в этой области было испробовано множество способов решения этой проблемы, один из которых — генерация данных на основе моделирования. Но есть еще одно узкое место, которому уделяется меньше внимания, но которое не менее важно: медленный цикл разработки моделей.

От того, насколько быстро и надежно мы можем проводить абляции, оценивать возможные варианты и сравнивать контрольные точки модели, зависит скорость итераций. Когда каждое решение приходится проверять с помощью реальных экспериментов, масштабировать исследования сложно. Моделирование — наш ответ на эту проблему. Это не отдельный алгоритм или программный компонент, а способ перенести наше понимание роботизированной системы в программу, которую мы можем запускать, проверять и совершенствовать.

From a pure model-learning perspective, simulation is often treated as a data generator. For us, however, it is much more than that: simulation is the infrastructure layer that accelerates robotics development.

Why We Start with Evaluation First Before Data Generation

Using simulation to solve the robotics data bottleneck is a compelling vision, and it was one of our original motivations when we started the Genesis World effort. After studying the problem more deeply and systematically, we realized a few things:

- Evaluation is the bottleneck we have to address first no matter what approach we use. Trustworthy simulation is the prerequisite for everything else: whether the goal is evaluation, data generation, or post-training RL, simulated behaviors must first match real-world behaviors at the system level.
- In the short term, real-world data collection has turned out to be economically viable for the scale and diversity we need, and is sufficient to reveal early scaling behavior. That gives us room to close the sim-to-real gap properly before relying on simulated data for training.
- Simulation provides an environment for data generation, but generating useful data requires several additional yet critical ingredients. To automate this process, we built an automated pipeline for task generation, reward specification, and data production through task-and-motion planning or reinforcement learning. The resulting data is meaningful, but still requires significant work to align its distribution with the deployment distribution.

Choosing simulation-based evaluation as our entry point lets us systematically study and reduce the sim-to-real gap, before addressing the more complex second-order effect of simulation for training data generation. It also accelerates model iteration, and lays a solid foundation for scaling up simulation later.

Simulation Compresses the Two Slowest Cycles:

Model Evaluation and Learning from Experience

Building a trustworthy and efficient evaluation system is a first-class problem on its own. At this stage, our goal is to establish strong correlation between simulation and the real world without relying on *any* simulation data in pretraining. The reason is simple: when training and evaluation share the same simulated distribution, an improvement could reflect either a truly better model and data recipe, or it could just be a result of a tighter fit to the simulator dynamics. Keeping

the pipelines separated gives us a cleaner signal about which experiments actually improve model performance.

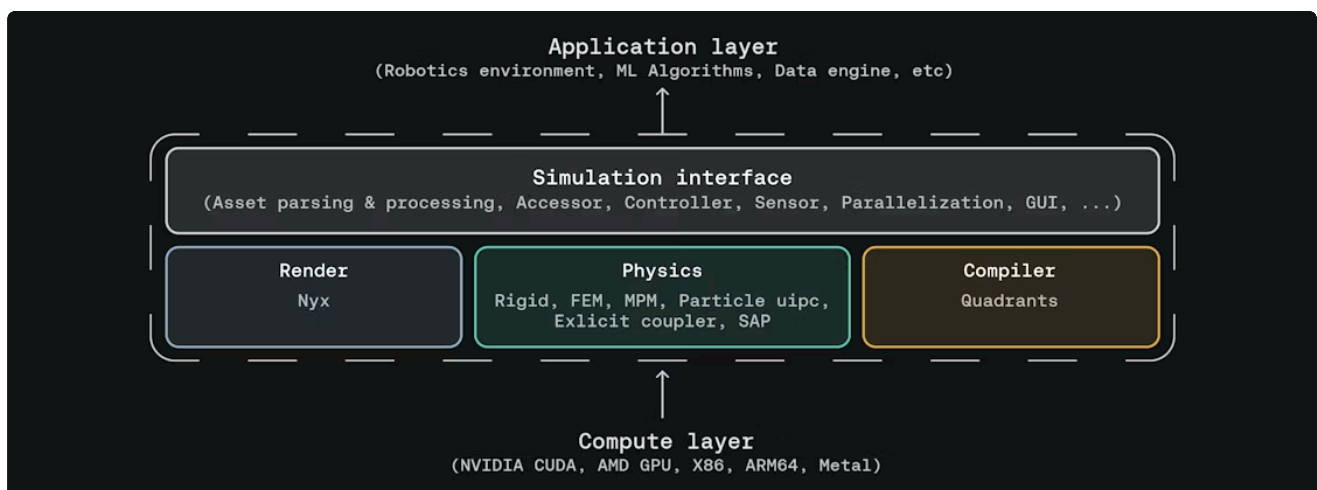
This is not a permanent rule. Sim-to-real RL has produced strong results in humanoid control, and we see scaling up simulation environments as a promising direction for larger-scale post-training of robotics foundation models. But before bringing any simulation data into training, we spent the past year developing a principled way to verify and improve the simulator's trustworthiness.

Simulation evaluation also doubles as a quantitative measure of the simulator itself: when a model's behavior and performance match between sim and real, the gap is small enough to trust for that model. Achieving this match requires tuning every layer of the stack, including hardware and system identification, control, compiler, physics, assets, and rendering.

Over the past year, evaluation has evolved from a noisy and time-consuming bottleneck to a deterministic compute problem that runs two orders of magnitude faster. Learning from experience comes next: a scalable environment where policies perceive and act, fail and recover, and continuously improve without risking damage to hardware or the surrounding environment.

The platform we built to make the above discussed points possible is Genesis World 1.0:

- [Nyx](#), our real-time photorealistic rendering engine designed from the ground up for robotics applications.
- [Quadrants](#), our cross-platform compiler for high-performance infrastructure.
- [Genesis World](#), our simulator with high-fidelity unified rigid and deformable physics.
- Simulation interface, the tooling that makes the engine easy to use for downstream applications.



The Evaluation Bottleneck

You can only improve what you can measure, and evaluating robotics foundation models is *expensive*. A model has to work across tasks, objects, and conditions. Evaluation also needs to surface failure modes, because those failures drive development priorities and data collection requirements.

A strong evaluation system must therefore be both *scalable* and *closed-loop*: scalable to cover the combinatorial space of tasks and conditions, and closed-loop to exercise the full perception-to-action pipeline rather than relying solely on offline metrics over static datasets. It also needs to run continuously and reproducibly throughout development. The autonomous-driving communities learned this early: teams that built scalable, closed-loop evaluation pipelines pulled ahead. Waymo already drives around 20 million miles a day in simulation since years ago, GM also simulate roughly 100 years of human driving every single day before public-road testing.

Robotics does not yet have mature infrastructure for large-scale simulation evaluation. In real world, we face the same pressure with worse constraints: robots aren't yet deployed at the scale of cars, and the task distribution is far broader. Even with auto-reset infrastructure, VLM critics, and minimal human intervention, real-world evaluation is capped by hardware availability, physical space, and the cost of running robots: expensive, noisy across stations and time, and mechanically incapable of reaching the breadth foundation models demand.

Today, a typical model evaluation at Genesis runs across hundreds of tasks, with each task repeated for hundreds of episodes. In the real world, that would add up to more than two hundred hours of continuous operation with one operator and one robot station for just a *single* evaluation pass. Statistically meaningful comparisons across checkpoints require many such passes.

In simulation, the same tens of thousands episodes:

- run in less than 0.5 hours, two orders of magnitude faster than real world;
- require no human or hardware in the loop;
- produce bit-exact result consistency across runs.

Once evaluation becomes cheap and continuous, every candidate policy, experimental branch, and hyperparameter change can be scored automatically against the full suite. Concretely, this gives us:

- *More experiments, better decisions.* More design choices can be validated before committing to large-scale pretraining, saving significant GPU compute.
- *Systematic coverage.* We can sweep variants that are nearly impossible to vary on hardware, including object shapes, surface materials, lighting angles, and camera trajectories, across *~10 axes* that reveal exactly where a policy breaks. We can also evaluate the same model across different end-effector form factors, from parallel grippers to different robotic hand designs.
- *Scalable iteration at organization-scale.* Every code update can be scored against a consistent benchmark, without waiting on operator schedules, robot availability, or local lab access. As our team grows, this lets more people contribute in parallel across continents, while still optimizing against the same objective.

- Reproducibility by default. Same scenario, same result, every time. No calibration drift, no wear and tear, no variation between operators.

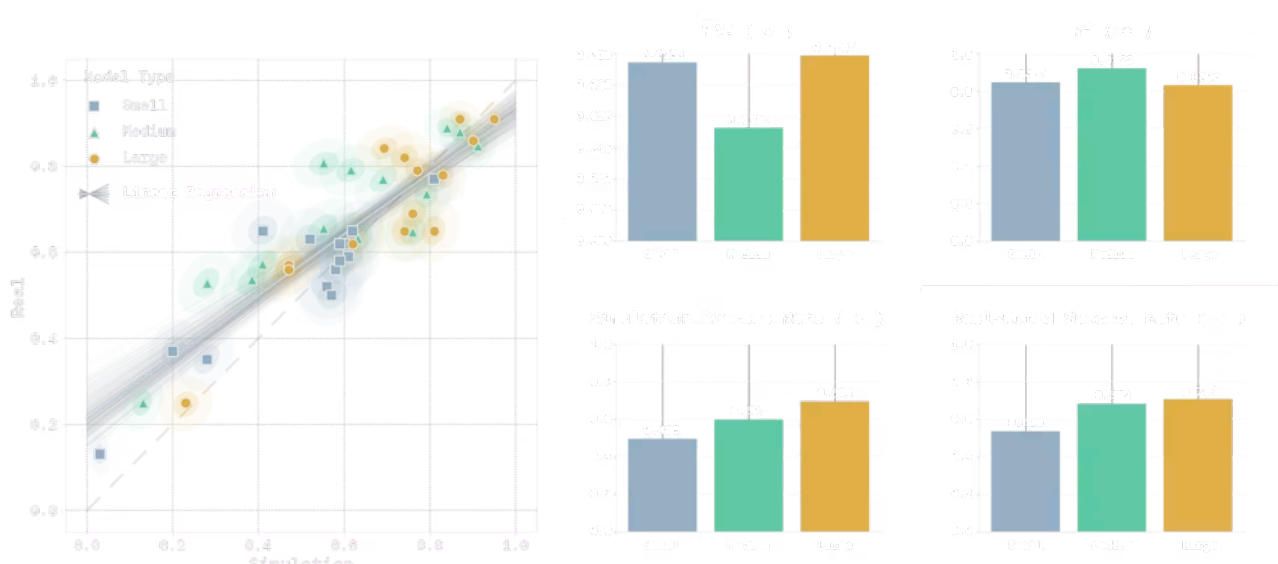
Can We Trust the Result?

The bar we held ourselves to is **zero-shot real-to-sim**: policies we evaluate in simulation are trained on real-world data only, keeping the training and evaluation workflows decoupled. The sim-to-real gap can come from several layers:

- Visual fidelity: material properties, lighting models, and camera characteristics tuned to match our real sensory pipeline.
- Robot kinematics and dynamics: precise modeling of joint behavior, friction, and contact.
- Low-level control: faithful replication of the actual controller running on our hardware, including timing, latency, and communication characteristics.

To identify the gaps, we built a comprehensive telemetry system and a real-time side-by-side rig that runs the simulator and the physical robot in parallel from the same initialization. This rig lets us choose the source of the policy inputs independently: observations such as camera frames and proprioception can come from the simulator, from the robot, or from a tunable blend of both. By swapping one component at a time and observing where divergence appears, we can attribute sim-real gaps to specific layers: physics, rendering, communication, or control, instead of collapsing them into a single binary success/failure outcome.

After this work, simulation evaluation correlates with on-hardware rollouts at 89%, and our reality gap is 45% smaller, measured by FID score on our dataset, than the next-best alternative simulator.



As an example, we evaluated three models with different scales and architectures, denoted as Small, Medium, and Large. We selected 14 tasks and ran both real-world and simulation evaluations, with 200 episodes per task. We then computed correlation metrics and applied 1,000,000 bootstrap iterations to estimate confidence intervals. In the correlation plot above, each data point is visualized with its bootstrap distribution, and 500 linear regression lines sampled from the same process are overlaid to reflect uncertainty in the correlation estimate.

For our primary evaluation, we focused on the Pearson correlation and the MMRV (Mean Maximum Rank Violation, proposed in [SimplerEnv](#)⁰¹). The results are strong: the Pearson correlation reached 0.8996 (95% CI: [0.7439, 0.9314]), demonstrating that our simulator faithfully reflects real-world performance trends. At the same time, the MMRV was low at 0.0166 (95% CI: [0.0102, 0.0474]), indicating that our simulator preserves the performance rankings of different models. When we compare the open-loop evaluation metrics (R-squared and Mean Absolute Error of action prediction on a fixed dataset) across these models, the open-loop scores do not reflect differences in real-world performance. Open-loop metrics are useful for catching spikes and as a sanity check, but once they fall within a narrow band, differences between models become indistinguishable in open-loop terms, while closed-loop metrics become much more informative.

We build digital twins not just as a 3D model of a workspace, but as a faithful replication of every layer of our stack, from actuator dynamics to pixel rendering. With enough engineering attention to the lowest-level details, the sim-to-real gap can be made virtually minimal. That same attention also reveals which parts of the robot stack matter most, and how.

Genesis World 1.0: Making Simulation More Trustworthy

To make simulation trustworthy, and to expand its coverage across the tasks we care about, our simulation platform has undergone a ground-up overhaul.

Nyx: A Purpose-Built Rendering Engine

Robotics does not get the renderer it needs from off-the-shelf tools. Every renderer is shaped by its target use case. Game engines optimize for visual appeal and often rely heavily on baking. Offline renderers go in the opposite direction: physically accurate, but often minutes per frame, with little room for scenario-specific optimization. Neither fits what robotics needs: millions of frames that look like what a real camera sees, generated fast enough to evaluate policies at scale, in an engine we can continue to extend.

To this end, we built *Nyx* to combine the best of both worlds: path-traced accuracy as the baseline, with rasterization shortcuts taken only where they do not compromise what downstream models learn from the image.

Three criteria shape our design:

- **Efficiency.** We aim to render noise-free 1080p frames in 4 ms or less on a high-end consumer GPU, with *no baking* and *no ghosting*. To get there, we use a visibility buffer, a bindless GPU-driven architecture, MSAA, hardware ray tracing, hardware matrix cores, and video compression, all tuned for GPU occupancy. The shortcuts we take are chosen to preserve the visual signals that matter to the policy.

- Minimal sim-to-real gap. Closing the gap is a property of the entire rendering stack. Owning Nyx end-to-end makes light transport, lighting, geometry, and the camera model intentional choices rather than inherited defaults, allowing us to optimize for visual realism rather than visual flourish. Path tracing is our baseline: multi-bounce lighting, soft shadows, and indirect illumination are correct by construction, with a physically grounded camera model on top so what the policy sees matches what a real sensor captures. Real-world data enters wherever possible: an HDRI pipeline lights scenes with measured radiance, and assets come from internal scanning and photogrammetry rather than authored stand-ins. 3D Gaussian splats extend this principle where mesh reconstruction falls short. The harder problem we focus on is reconciling image-based lighting with splat-based geometry, so captured assets participate correctly in path-traced light transport.
- Tight Genesis integration. Nyx plugs directly into Genesis and is driven by batched physics rather than scene-by-scene execution. This lets us run thousands of parallel rollouts, each with its own scenario, lighting, and camera trajectory, rendered through a single unified pipeline. That coupling is what turns rendering throughput into evaluation throughput.

Unified Physics

Real-world manipulation is rarely about one mode of physics at a time. Genesis runs multi-physics in a single pipeline: articulated rigid bodies (MJCF/URDF/USD) of different embodiments as shown in our video: Wuji, Sharpa, Genesis hand, Pika gripper, Tianji arm, G1 humanoid, etc. FEM for elastic deformables and cloth, MPM for granular and elasto-plastic materials, SPH for fluids, and PBD for fast cloth and position-based liquids.

We designed three interchangeable couplers behind the same scene API: a fast general-purpose coupler; a Drake-style Semi-Analytic Primal coupler⁰² with hydroelastic contact; and an Incremental Potential Contact (IPC) coupler with intersection-free contact for delicate deformables. Switching between different couplers can be done via a one-line change, without change in assets, sensors, and the policy interface. Heterogeneous parallel simulation mixes different objects, kinematic trees, and scene layouts in a single batched environment.

To more tightly couple IPC with articulated robots, we extended libuipc⁰³ with an External Articulation Constraint that embeds joint-space dynamics directly into IPC's optimization, so joint-space forces and contact forces resolve simultaneously rather than staggered between separate solvers. For an articulated system with m joints, the rigid solver predicts joint displacements $\tilde{\boldsymbol{\theta}}$ and computes the joint-space effective mass matrix $\mathbf{M}^t$, injected into IPC as an external articulation kinetic energy:

$$K = \frac{1}{2} \left(\delta\boldsymbol{\theta}(\mathbf{q}, \mathbf{q}^t) - \tilde{\boldsymbol{\theta}} \right)^T \mathbf{M}^t \left(\delta\boldsymbol{\theta}(\mathbf{q}, \mathbf{q}^t) - \tilde{\boldsymbol{\theta}} \right)$$

where $\delta\boldsymbol{\theta}$ maps IPC affine-body states $\mathbf{q}$ to joint-space displacements. IPC minimizes this jointly with contact barriers, friction, and joint constraints. Without contacts, the solver recovers the articulated prediction exactly; with contacts, it deviates just enough to resolve them, weighted by effective mass so heavier links resist correction more.

On the contact-handling side itself, we also developed barrier-free elastodynamics⁰⁴ to accelerate IPC-style simulation in contact-heavy scenes. Standard IPC enforces non-penetration with a logarithmic barrier, which both makes the Hessian ill-conditioned for tight contact and slows down active set exploration due to the filtered line search. We replace the barrier with a custom augmented Lagrangian: every requisite contact pair returned by continuous collision detection enters the active set immediately, and constraint satisfaction is driven by adaptive Lagrange multiplier updates rather than escalating penalty stiffness. For each contact pair i with current linearized penetration depth $c_i(\mathbf{x})$, we first introduce a slack variable s_i to convert the non-penetration inequality constraint $c_i(\mathbf{x}) \geq 0$ to an equality one: $c_i(\mathbf{x}) - s_i = 0$. Then, the per-step objective is defined as

$$L(\mathbf{x}, \mathbf{s}, \boldsymbol{\lambda}) = E(\mathbf{x}) + \sum_{i \in \mathcal{A}} \psi(c_i(\mathbf{x}), s_i, \lambda_i, \mu),$$

where E is the incremental potential, $\mathcal{A}$ is the active contact constraint set, and ψ is the augmented-Lagrangian terms with stiffness μ and Lagrange multiplier λ_i . After each primal solve that alternates between

$$\mathbf{x} \leftarrow \arg \min_{\mathbf{x}} L(\mathbf{x}, \mathbf{s}, \boldsymbol{\lambda}) \quad \text{and} \quad \forall i, s_i \leftarrow \max(0, c_i(\mathbf{x}) - \lambda_i/\mu),$$

we update the Lagrange multipliers via

$$\lambda_i \leftarrow \lambda_i - \mu(c_i(\mathbf{x}) - s_i),$$

and then update $\mathcal{A}$ (as well as $c_i(\mathbf{x})$) to keep it compact while effective. The Hessian stays well-conditioned even as the stress increases, and contact-rich benchmarks run up to 103× faster than traditional IPC in complex scenes while still guaranteeing no intersections.

Beyond unified physics, we matured the engine along three axes:

- **Speed at scale.** Cooperative threading in linesearch, GPU graph in the decomposed solve, tile-blocked Hessian factorization, broadphase optimizations, register-only Cholesky and solver tiles, and a narrowphase optimized for minimum thread divergence and maximum GPU core utilization. Rigid simulation of complex scenes now runs significantly faster, parallel simulation is extended to deformables and path planning, and many optimizations that were previously CUDA-only have been supported on other GPU backends as well.
- **Numerical stability.** Inertial-axes alignment for free-joint stability, auto-calibrated solver tolerance, safe GJK fallback, noslip slip/drift suppression, and a unified line-search path across decomposed and monolith solvers. Long-standing edge cases in USD/MJCF/URDF parsing, compound-joint Jacobians, box-box and MPR collision, IK quaternion singularities, and per-platform stability have been resolved.
- **Coverage.** Point-cloud tactile, temperature-grid, and proximity sensors are now supported alongside the existing FOTS elastomer-displacement, magnetometer-IMU, and contact-probe suite. We extended our solver set with Implicit FEM (Newton + CG) and linear

corotated elastics. Our asset support now extends to URDF xacro, MuJoCo general actuators, compound/mimic joints, and equality/weld constraints. Public APIs cover vertex manipulation, kinematic/potential energy queries, FK, Jacobian-at-point, and mass-matrix access. With this coverage in place, we built cross-embodiment simulation environments spanning Wuji, Sharpa, Genesis hand, and Pika gripper, supporting both soft- and rigid-body manipulation tasks.

Quadrants: Cross-Platform Compiler for GPU-Accelerated Computation

The same physics pipeline has to run on the robot's onboard computer, engineers' MacBooks, and our GPU clusters without forking code per target. We achieve this with Quadrants, our compiler for GPU-class workloads.

Quadrants began as a fork of Taichi⁰⁵ roughly a year ago, and we gave it this name to reflect its Taichi origin. In Quadrants, kernels are written in plain Python and JIT-compiled to NVIDIA (CUDA), AMD (ROCm), Apple Metal, Vulkan, and x86/ARM64 CPUs via LLVM. Since the fork, we have significantly rebuilt the parts that matter most for simulation workloads, achieving up to 4.6x faster runtime on our manipulation and locomotion benchmarks.

To stay efficient on each backend, the compiler maps SIMT primitives at the subgroup and block level to the native equivalent (warps of 32 on NVIDIA, waves of 64 on AMD, subgroups on Metal), so a hand-tuned contact solver runs without per-platform branches. Reverse-mode autodiff, previously experimental, is now a first-class citizen on every backend, making differentiable simulation portable across the same hardware our policies are deployed on. We also added a pure-Python backend for easier debugging and systematic coverage testing.

At simulation scale, the cost shifts from per-kernel compute to the overhead of orchestrating many small kernels per physics step, and Quadrants attacks that overhead from several angles. We record each physics step as a single kernel graph, hardware-accelerated on CUDA (with conditional loops on SM90+) and supported in software on every other backend, which removes launch latency from every top-level for-loop. Independent kernels can overlap through streams, running in parallel on the same GPU rather than serializing through one queue. Where launches do remain, hand-tuned kernel-launch contexts cached at multiple memory tiers keep dispatch overhead sub-microsecond even when many small kernels fire back to back.

Inside kernels, dense linear algebra (Cholesky factorization, triangular solves) is expressed with intuitive Python syntax that compiles to 16×16 tile-blocked code paths, and whole-kernel common-subexpression elimination catches redundant work that block-local optimizers miss. A lightweight perf-dispatch layer benchmarks kernel variants on first call for a given argument geometry and caches the fastest choice per signature, so the same frontend code adapts to whatever hardware it runs on.

Around all of this is a three-layer cache for compiled artifacts: compiled kernels on disk, plus PTX and fast-cache layers for process startup. Scene switches reuse cached kernels rather than triggering recompilation; CI and iteration runs start almost instantly, a more than 10× speedup that cuts startup time from minutes to seconds.

On the data side, tensors come in two interchangeable types: field for peak runtime throughput, and ndarray for fast startup and compile time. We designed a unified wrapper to switch between them, or between physical layouts, at *runtime* without changes to the calling code. Our kernels accept nestable Python dataclasses containing both types as arguments, so simulator

bookkeeping stays in plain Python rather than in parallel struct definitions. For interop with the ML stack, tensors share device memory with PyTorch via DLPack, and on Metal we share a command queue with PyTorch so zero-copy does not introduce synchronization overhead.

Scene and Asset Pipeline

Useful simulation needs both faithful digital twins of real workspaces and a long tail of diverse object assets. We have two complementary pipelines to feed the simulator.

Our photogrammetry pipeline turns multi-view captures, collected through our in-house iOS app, a digital camera, or off-the-shelf devices with VIO poses as initialization, into accurate 3D maps, then trains meshes and Gaussian splats end-to-end from the raw images and poses. Both feed Nyx for rendering and Genesis for physics.

On top of reconstruction, we explored a programmatic pipeline to generate simulation environment, including scene layout, asset selection, environment code, and success metrics, so complex environments can be built automatically.

Below is an example where we reconstruct a scene, place robots into it, execute tasks, and emulate a variety of sensors (RGB, depth, tactile, lidar, etc.). With good assets and tasks, simulation can deliver a wide range of sensor modalities that are otherwise hard to obtain in the real world.

Systematic Evaluation

With the simulator, renderer, compiler, and asset pipeline all in place, evaluation can do what real-world testing cannot: probe a policy along each dimension of robustness, at a scale and frequency that real-world hardware cannot support.

Useful evaluation is more than a single scalar number: a policy that scores 80% on a nominal-setting benchmark may still collapse under a lighting change, a camera shift, or a rephrased instruction. We refer to prior work⁰⁶ and structure our evaluation as a taxonomy of orthogonal perturbation axes, each designed to stress-test a specific category of model understanding. Because we run these evaluations in simulation, we can test at a scale and frequency that would be impractical on physical hardware:

- Visual: lighting conditions, camera perturbation, background variation
- Behavioral: unseen combination, object placement, robot configuration

- Semantic: language rephrasing, subtask ordering, camera viewpoint

For each axis, we vary a single parameter while holding all others at their nominal values. These failure modes guide us what data to collect to make our model more robust. We also use this framework to compare models. We define robustness on a given axis as the relative performance retention under perturbation compared to the nominal, unperturbed setting. We measured their per-axis robustness profiles, and tracked how these profiles evolve with cumulative training FLOPs.

“Put the tomato into the green bin to clear the table.”

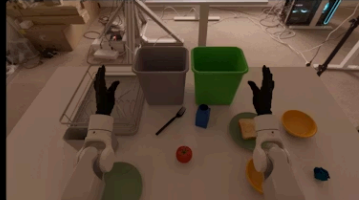


Visual

Background, Lighting, Camera Pose, Distractor



Background



Lighting



Camera Pose



Distractor

Behavioral + Visual

Object Scale



Object Scale

Behavioral

Unseen, Combination, Unseen Object



Object Pose



Robot Height

Behavioral + Semantic

Robot Height, Object Pose, Friction, Reactivity

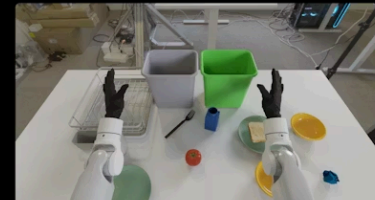


Unseen Object:

“Put the carrot into the green bin to clear the table.”

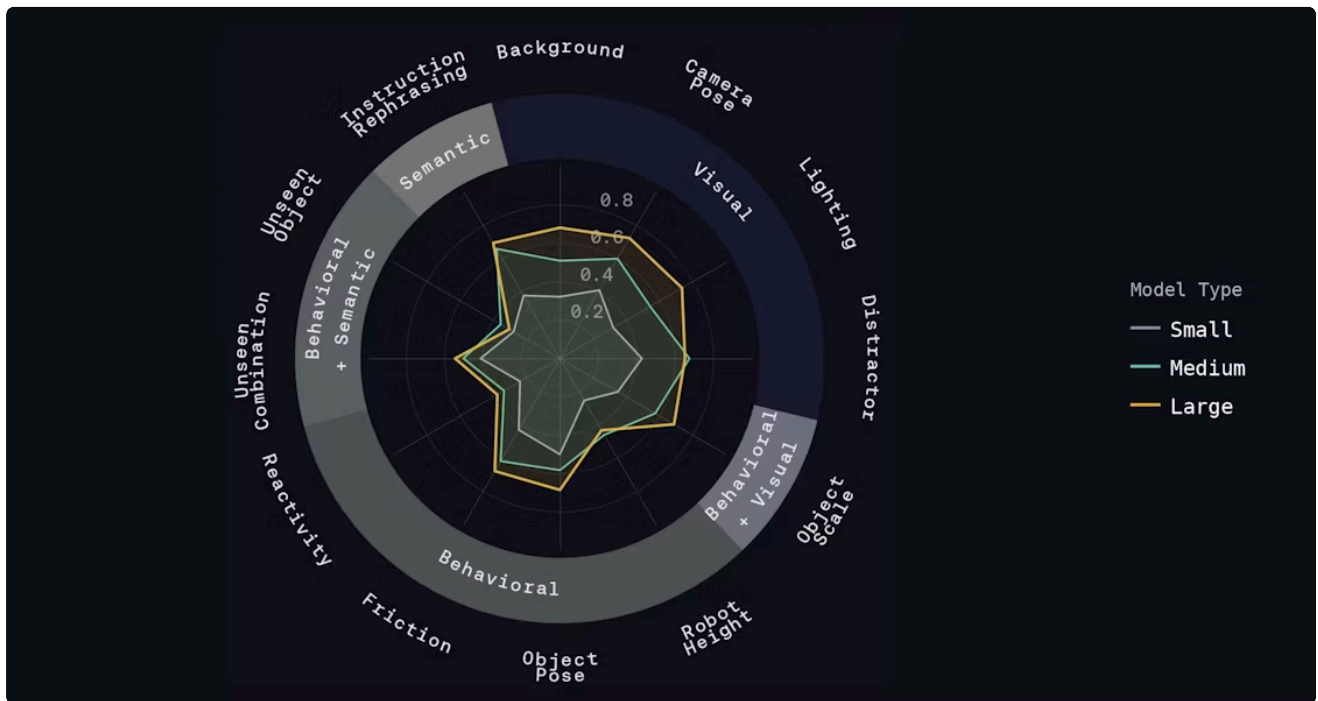
Semantic

Instruction, Rephrasing



Instruction Rephrasing:

“First, throw away the tomato into the green recycle bin.”



These profiles surface capability differences across models that aggregate success rates hide, and they identify which axes need additional data collection to improve robustness. Our perturbation sweeps across training checkpoints at multiple model scales requires thousands of evaluation episodes per data point, and is only feasible when evaluation itself is nearly free.

Simulation for Robotics: The Path Forward

Once a system can be evaluated at scale, compute can take it over and push it to the limit. Going forward, we are focused on three directions.

Scaling post-training by scaling simulation environments. Closed-loop evaluation can also become a data engine for exploration: the model attempts tasks, fails, is scored, and improves across millions of iterations and thousands of tasks in parallel, with the simulator acting as both environment and critic. Large-scale RL in simulated sandboxes has become a key ingredient of LLM development, and we are scaling environments for robotics with the same playbook.

Hybrid simulator. Genesis World today is classical and heuristic, giving us full controllability and observability over every layer of the stack: every knob is explicit, and every behavior can be inspected, modified, and attributed. Learned simulators are progressing quickly, but much of their state remains implicit and ungrounded, which is where classical simulators remain stronger today. We are working to merge the two in a data-driven way, using the diverse multimodal data we have been collecting. Classical simulation brings grounding; learned world models bring scale and realism. Over time, the boundary will blur, and both halves will reinforce each other.

Self-evolving physical AI as our north star. In the digital AI world, well-built harnesses with environment allows models and agents to improve increasingly on their own. For physical AI,

simulation can serve as that harness layer. Our north star is self-evolving physical AI: improvement running end-to-end through agents across two loops. The inner loop runs in simulation, where agents generate environments, the model acts, the simulator scores, and the policy improves. The outer loop runs in the real world, where deployments surface edge cases that recalibrate the simulator, and agents fold those cases back into the task distribution. Every variable a researcher would normally touch, including data mixture, architecture, reward, curriculum, and new environments, can be tuned by agents as long as each change is verifiable. All of this serves one goal: keeping robotics foundation model development from being bottlenecked by the $1\times$ speed of the real world. With a trustworthy simulator, the throughput of ideas scales with compute, and each development cycle becomes dramatically shorter.

Across all three directions, the idea is the same: simulation is not merely a data generator for robotics. It is a fundamental infrastructure layer. Once simulation is embedded into the development process, progress can move at the speed of compute, rather than being limited by real-world wall-clock time.

More to come soon.

Citation

If you found this work useful in your research, consider citing it as:

```
@article{
  genesis2026genesisworld1,
  author = {Genesis AI Team},
  title = {The Role of Simulation in Scalable Robotics, Genesis World
1.0, and the Path Forward},
  journal = {Genesis AI Blog},
  month = {May},
  year = {2026},
  url = {https://www.genesis.ai/blog/the-role-of-simulation-in-
scalable-robotics-genesis-world-10-and-the-path-forward},
}
```

